

Actuate

Architecture Design Document

*A Production-Grade, Multi-Rig, Sensor-Fused Data Engine for
Zero-Touch Robot Foundation Model Training*

Prepared as an internal architecture reference for DatraAI's Actuate pipeline. Scope: egocentric head-mounted capture, UMI handheld gripper rigs, calibrated stereo rigs, instrumented-glove capture, and teleoperated robot data — explicitly **not** internet/web-scale video mining. Target bar: a customer (e.g. a frontier humanoid or manipulation lab) should be able to take delivered data and begin policy training with zero additional processing.

Contents

1	Executive Summary & Design Mandate
2	Why the EgoInfinity Model Doesn't Directly Transfer
3	Capture Rig Taxonomy
4	Layer 0 — Multi-Rig Ingestion & Canonicalization
5	Layer 1 — Perception & Metric Reconstruction
6	Layer 2 — Sensor-Fused Interaction Refinement
7	Layer 3 — Canonical 4D Multimodal Representation
8	Layer 4 — Quality Certification & Fail-Closed Gating
9	Layer 5 — Cross-Embodiment Retargeting Engine
10	Layer 6 — Language & Task Annotation
11	Layer 7 — Dataset Packaging & Delivery
12	Layer 8 — Feedback Loop & Continuous Improvement
13	Infrastructure & Scaling
14	Comparative Positioning
15	Appendix — Schemas, Thresholds & Coordinate Conventions

1 Executive Summary & Design Mandate

Most robotics data pipelines stop at the point of producing *labeled video*. That is necessary but not sufficient. The mandate for this architecture is stricter: a customer — a humanoid or manipulation-focused robotics lab building generalist policies — should be able to take a delivered dataset and begin training without writing a single additional line of preprocessing code. This document specifies the full pipeline that makes that guarantee possible, end to end: from heterogeneous raw capture across five distinct rig types, through metric-scale perception, sensor-fused interaction understanding, cross-embodiment retargeting, certification, and delivery in whatever schema the customer's training stack already expects.

The central design principle is **sensor-truth priority**. Unlike systems built to mine uncontrolled internet video — where no ground-truth sensor stream exists and every quantity must be estimated visually — this pipeline is built for *controlled capture*: head-mounted rigs, UMI handheld grippers, stereo depth rigs, instrumented gloves, and teleoperated robots, each of which can carry real IMU, encoder, aperture, or force sensors alongside the camera. Wherever a real sensor signal exists, it is treated as ground truth and vision is used to fill gaps, not the reverse. This one architectural choice is what allows this pipeline to exceed the physical reliability of a pure-vision engine, at the cost of requiring calibrated hardware rather than arbitrary found footage.

What "training-ready" means here

- Every episode carries metric-scale 3D geometry for hands, grippers, and manipulated objects — not 2D pseudo-actions.
- Every episode carries a machine-checkable **quality certificate**: per-modality confidence, calibration completeness, kinematic feasibility, and an explicit retargeting-eligibility flag per target embodiment.
- Every episode is retargeted, not just recorded — delivered as executable joint trajectories for the customer's actual robot morphology, validated in simulation before delivery.
- Delivery format is a customer-facing concern, not a pipeline concern: the same canonical representation exports to LeRobot v3, RLDS, or a bespoke schema without touching upstream stages.
- Nothing ships without consent and PII redaction resolved — this is a hard, fail-closed gate, not a checklist item.

Document scope

Sections 3 through 12 walk the pipeline layer by layer, in the order data actually flows. Section 2 is deliberately placed first because it explains *why* several design choices here diverge from the closest published reference architecture (EgoInfinity) — understanding that divergence is necessary context for every layer that follows.

2 Why the EgoInfinity Model Doesn't Directly Transfer

EgoInfinity (Wang et al., 2026) is the strongest published reference for this problem class: a fully automated, modular engine that lifts in-the-wild internet RGB video into metric 4D hand-object representations, with a functional cross-embodiment retargeter validated on real hardware. Its core techniques — cross-module metric calibration, interaction-state-driven trajectory refinement, and an SE(3)-equivariant root-frame estimator trained entirely in simulation — are directly relevant and are adapted throughout this document. But its governing constraint is different from ours, and that difference cascades through nearly every layer.

The core constraint difference

Dimension	EgoInfinity (web video)	This pipeline (controlled capture)
Data source	Uncontrolled internet footage, arbitrary viewpoint	Purpose-captured: head-mount, UMI, stereo, glove, teleop robot
Ground-truth sensors	None — everything estimated visually	Real IMU, gripper aperture encoders, glove joint sensors, robot joint encoders where applicable
Camera assumption	Approximately static camera; excludes body-mounted footage	Explicitly includes moving, body-mounted, and handheld capture as a first-class case
Grasp-state detection	Vision heuristics only (2D mask overlap, 3D fingertip proximity)	Real gripper aperture / glove flex-sensor signal as primary source, vision as fallback
Object identity	Auto-discovered from noisy video captions	Can be operator-tagged at capture time and cross-checked against vision
Scale	Bounded by size of internet corpus, diversity-optimized	Bounded by capture program throughput, precision-optimized
Delivery target	Research artifact + interactive browser	Commercial delivery in customer's exact training schema, with certification

What we keep from EgoInfinity

- The **modular, component-replaceable** engine philosophy — every perception module (hand mesh, depth, segmentation) is swappable as stronger foundation models emerge.
- **Cross-module metric calibration** as a first-class step, not an afterthought — aligning every module's output into one shared scale and camera frame before any downstream reasoning.
- The **interaction-state-driven** refinement pattern (static / grasped / moving) as the backbone of trajectory stabilization, extended in Section 6 with real sensor overrides.
- The **SE(3)-equivariant, flow-matching root-frame estimator** for cross-embodiment retargeting (Section 9), trained in simulation exactly as EgoInfinity does, but conditioned on rig type.
- The discipline of reporting **kinematic feasibility, contact consistency, and real-robot validation** as the actual bar for "does this work," not just perceptual accuracy.

What we change or add

- **Sensor fusion arbitration** (Section 6) — a trust hierarchy where real sensor ground truth (gripper aperture, glove joint angle, force/tactile) overrides vision-only estimates whenever present, rather than relying on vision heuristics unconditionally.
- **Rig-aware ingestion** (Section 4) — five structurally different raw formats must be normalized into one schema before any perception model runs, a problem that does not exist when every input is an arbitrary MP4.
- **Moving-camera support as default**, not an excluded case — head-mounted and handheld UMI capture are the primary sources here, not an edge case to defer.
- **A formal quality certificate per episode** (Section 8), extending EgoInfinity's sanity-check philosophy into a customer-facing, machine-readable trust artifact.
- **Customer-format delivery as a pluggable adapter layer** (Section 11), since EgoInfinity's output target is a research browser, not a specific customer's training loader.

In short: EgoInfinity answers "how do we extract robot-usable signal from video we don't control." This pipeline answers "how do we extract the highest-fidelity signal possible from capture we do control, and prove it's trustworthy enough to train on without human review." The techniques transfer; the trust model does not.

3 Capture Rig Taxonomy

The pipeline must ingest five structurally distinct capture configurations. Each is described below with its native channels — this is the input contract every downstream layer is built against.

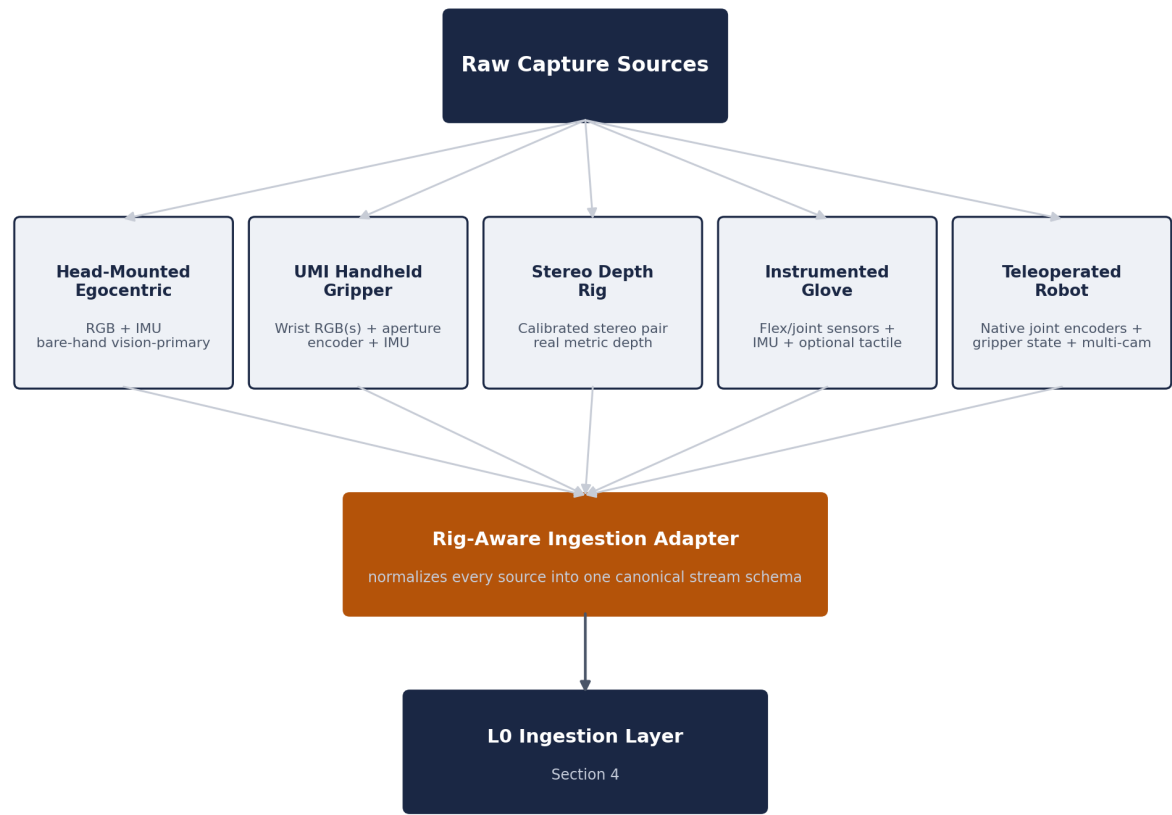


Figure 3.1 — Five capture rig types converge through a rig-aware ingestion adapter into one canonical stream schema.

Rig type	Native channels	Primary use
Head-mounted egocentric	RGB (mono or stereo), head IMU, optional audio	Bare-hand manipulation at scale; broad task/environment diversity
UMI handheld gripper	Wrist-mounted RGB (fisheye, 1+ cameras), gripper aperture encoder, wrist IMU	Gripper-realistic manipulation without full robot hardware
Stereo depth rig	Calibrated stereo RGB pair, real metric depth stream	Highest-confidence 3D reconstruction; calibration reference set
Instrumented glove	RGB, per-finger flex/joint-angle sensors, glove IMU, optional tactile array	Ground-truth finger kinematics independent of vision
Teleoperated robot	Multi-camera RGB(+D), native robot joint encoders, gripper state, end-effector force (if equipped)	Directly executable action labels; no retargeting required

Why this taxonomy matters architecturally

Every layer from L1 onward must declare, per rig type, which of its inputs are *estimated* versus *measured*. A head-mounted session has no grasp ground truth at all — grasp state must be inferred visually, exactly as in EgoInfinity. A UMI session has a real aperture encoder — grasp state is measured directly and vision serves only as a cross-check. This distinction is carried as a first-class field (`ground_truth_channels`) on every session from ingestion onward, and every downstream confidence score is computed differently depending on it.

4 Layer 0 — Multi-Rig Ingestion & Canonicalization

Ingestion's job is singular: take whatever a given rig natively produces and emit one common container format that every later layer can consume without knowing which rig the data came from. This is a harder problem here than in a single-rig pipeline, because the five rig types in Section 3 do not share a raw format, sample rate, or even coordinate convention.

4.1 Container format

Raw multi-channel capture is ingested into a per-session MCAP-style container — one session per recording, one topic per channel, each message carrying a hardware timestamp. This is the same design choice EgoInfinity makes for its output packaging and that the UMI gripper data specification (Section 7 of the vendor requirements this pipeline is designed to satisfy) mandates as raw provenance. Storing raw provenance in this form — rather than only the processed derivative — is what allows re-processing an entire historical corpus when a perception module improves, without re-capturing anything.

4.2 Canonical stream schema

Field	Type	Notes
session_id	string	Globally unique, immutable once assigned
rig_type	enum	head_mounted / umi_gripper / stereo / glove / teleop_robot
ground_truth_channels	list[enum]	Which of {grasp, depth, finger_pose, joint_state} are hardware-measured, not estimated
camera_streams[]	list	One entry per physical camera: intrinsics, extrinsics-if-known, resolution, native fps
imu_streams[]	list	Mount location (head / wrist / glove), sample rate, units
encoder_streams[]	list	Gripper aperture, joint angle, or robot joint encoder, as applicable
timestamp_basis	string	Shared clock domain all streams are aligned to
calibration_ref	string	Pointer to intrinsics/extrinsics file, or explicit "none — approximated"

4.3 Synchronization

All streams are resampled onto a single, shared timestamp axis at ingestion time, using hardware timestamps where the rig provides them (preferred) and interpolation against the highest-rate stream otherwise. Millisecond-level synchronization precision is enforced as a hard requirement carried into the Layer 4 quality gate — sessions whose native hardware cannot support this are flagged, not silently accepted at lower precision.

4.4 Calibration ingestion

Camera intrinsics and, where available, extrinsics are ingested per physical device, not per session — a rig's calibration is a property of the hardware, re-used across every session captured with it until the hardware is recalibrated. Sessions lacking a calibration reference fall back to an approximated pinhole model, and this fact is carried forward explicitly as a lowered-confidence flag rather than silently treated as equivalent to real calibration.

5 Layer 1 — Perception & Metric Reconstruction

This layer produces the raw perceptual estimates that Layer 2 will later cross-validate against real sensor data. Every component here follows EgoInfinity's precedent of using strong, swappable, off-the-shelf foundation models rather than training bespoke perception networks — the pipeline's value is in orchestration, calibration, and fusion, not in re-deriving hand pose estimation from scratch.

5.1 Metric camera and depth calibration

- **Stereo rigs:** real calibrated depth stream consumed directly — the highest-confidence source, used as ground truth for calibrating monocular estimates on other rigs captured in similar conditions.
- **Monocular rigs** (head-mounted, UMI, glove): a metric monocular depth/geometry model estimates per-frame depth and camera focal length, calibrated against any co-located stereo reference data where available.
- **Gravity direction** is estimated per session from a small set of representative frames, used later for retargeting root-frame estimation and for camera-motion-aware egocentric reframing.

5.2 Hand pose and mesh estimation

A metric hand-mesh estimator recovers per-frame hand pose, shape, and 3D keypoints from RGB. For instrumented-glove sessions, this vision estimate is *never treated as the primary signal* — it exists only as a cross-check against the glove's own joint-angle sensors (Layer 2). For all other rig types, it is the primary hand-kinematics source, infilled and smoothed exactly as a single-clip system would, with biomechanical joint-limit clamping.

5.3 Object discovery, segmentation, and tracking

Target objects are identified either from operator-provided task metadata (preferred, since capture is controlled, unlike EgoInfinity's caption-derived auto-discovery) or, when absent, from an open-vocabulary detector prompted by whatever task description is available. Detected objects are segmented and tracked through the clip by a promptable video segmentation model, and lifted into per-frame 3D point clouds using the calibrated depth from 5.1. Where sufficient unoccluded views exist, a single canonical object mesh is reconstructed once per object and reused across the clip.

5.4 Two-pass processing for throughput

Following EgoInfinity's scalability pattern: a lightweight first pass scans for hand/gripper presence and motion activity to filter out dead time (setup, idle stretches, camera repositioning), and the full reconstruction stack runs only on active segments identified by that pass. This keeps compute cost proportional to actual manipulation content, not raw recording duration.

6 Layer 2 — Sensor-Fused Interaction Refinement

This is the layer that most substantively departs from a pure-vision engine, and it is the core of this pipeline's reliability advantage. Pure visual object tracking is temporally unstable — poses drift for static objects, fail under occlusion, or fluctuate under weak visual correspondence, exactly as EgoInfinity documents. EgoInfinity's answer is to classify each frame into an interaction state (static / grasped / moving) using vision alone and let that state select which geometric signal drives the object pose. This pipeline keeps that state-machine pattern but changes what feeds it.

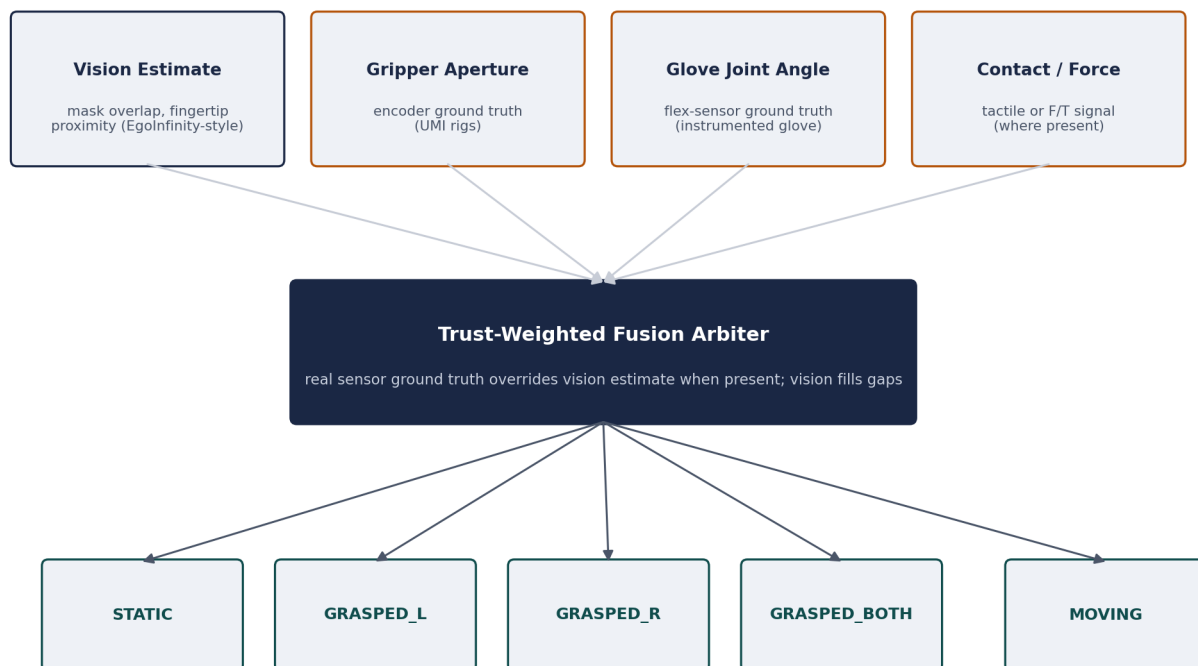


Figure 6.1 — Real sensor channels, where present, override vision-only estimates in the fusion arbiter before interaction-state classification.

6.1 Trust-weighted fusion arbiter

For every frame, each candidate signal — vision (mask overlap, fingertip proximity), gripper aperture encoder, glove joint angle, and contact/force sensing where equipped — reports both a value and a native confidence. The arbiter applies a fixed trust ordering: a hardware-measured grasp signal (aperture encoder closed beyond a threshold, or glove fingers curled beyond a biomechanical grasp posture) is authoritative and vision is used only to resolve ambiguity vision can uniquely see (e.g., *which* object is grasped, when multiple candidates are within reach). On rig types with no such hardware channel (head-mounted bare-hand), the arbiter degrades gracefully to EgoInfinity's vision-only heuristic, with the resulting lower trust tier explicitly recorded rather than silently equated with a sensor-confirmed grasp.

6.2 Interaction states

The refined per-object state machine mirrors EgoInfinity's six-state internal representation (static-global, static, grasped-left, grasped-right, grasped-both, moving), collapsed to the same three coarse states downstream (static / grasped / moving) for pose-source selection. The hierarchical assignment and dominant-hand resolution logic for ambiguous bimanual frames follows the same majority-vote-then-proximity-tiebreak pattern, with the trust-weighted signal from 6.1 substituted wherever a hardware channel is present.

6.3 Pose-source selection by state

State	Translation source	Rotation source
Static	Robust point-cloud centroid (outlier-filtered)	Canonical mesh orientation
Grasped (vision-only rig)	Hand-relative rigid bind, aggregated over the grasp segment	Same rigid bind
Grasped (sensor-confirmed rig)	Gripper/glove kinematic chain forward-computed from encoder + wrist pose	Same forward-kinematic chain
Moving	Smoothed mask-bbox back-projection	Per-frame PCA, sign-corrected

6.4 Sanity filtering

Two automated checks run regardless of rig type: a scale-sanity check that overrides an implausible monocular object scale with the mask-implied scale when they disagree beyond a fixed factor, and a spurious-detection check that flags (but does not delete) objects whose 3D position is inconsistent with any hand activity across the clip. Flagged objects are dimmed in the delivered representation, not silently dropped — the certification layer (Section 8) surfaces this rather than hiding it.

7 Layer 3 — Canonical 4D Multimodal Representation

Every rig type, after Layers 1 and 2, converges into one agent-agnostic representation. This is the single most important interface in the whole architecture: everything upstream is rig-specific, everything downstream (retargeting, annotation, packaging) is rig-agnostic and reads only this schema.

7.1 Per-frame episode state

Field	Description
hand_mesh / gripper_state	Full hand mesh + keypoints, or gripper aperture + wrist pose, depending on rig
hand_pose (SE(3))	Global metric pose of the acting end-effector (hand or gripper)
object_point_cloud	Per-frame back-projected, outlier-filtered point cloud
object_mesh	Canonical reconstructed mesh, one per object per clip
object_pose (SE(3))	6-DoF object pose in the shared metric camera frame
interaction_state	static / grasped / moving, plus dominant-hand / side resolution
ground_truth_provenance	Per-field flag: measured vs. estimated, and which sensor if measured
per_field_confidence	Numeric confidence per field, not just a single episode-level score

7.2 Coordinate conventions

All geometry lives in a right-handed camera-world frame, gravity-aligned using the Layer 1 gravity estimate. Object-canonical points are mapped into the camera frame via a fixed scale and rotation convention, matching EgoInfinity's coordinate handling so that any object reconstruction tooling built against that convention remains compatible.

7.3 Egocentric-consistent reframing

For sessions captured from a fixed or exocentric viewpoint (e.g., a third-person capture of a demonstration), the pipeline optionally performs a rigid, geometry-preserving reframing into a synthesized egocentric view — following EgoInfinity's exo-to-ego approach of re-rendering from a virtual camera anchored to hand activity, rather than a generative 2D video translation that risks pixel hallucination. This gives customers a consistent egocentric observation regardless of how the source footage was actually captured, without ever fabricating visual content.

8 Layer 4 — Quality Certification & Fail-Closed Gating

Nothing downstream of this layer should ever have to guess whether an episode is trustworthy. Every episode receives a machine-readable certificate before it is eligible for retargeting, annotation, or delivery. This is the layer that turns "we processed your data" into "here is exactly how much to trust every part of it."

8.1 Certificate composition

Component	What it measures
Sync integrity	Max/mean timestamp drift across all streams against the shared clock
Calibration completeness	Real vs. approximated intrinsics/extrinsics; gravity estimate confidence
Perception confidence	Aggregated per-field confidence from Layer 1/2, weighted by ground-truth provenance
Contact consistency	Cross-check between hardware grasp signal (if present) and vision-estimated contact
Kinematic feasibility	Fraction of frames for which downstream IK (Section 9) converges within tolerance
Retargeting eligibility	Per-target-embodiment boolean, driven by depth/calibration confidence thresholds
Consent status	Explicit granted / pending / denied — packaging is blocked until granted

8.2 Fail-closed delivery gate

Two independent gates must both clear before an episode is eligible for delivery: an overall quality score above a configurable floor, and an explicit consent grant. Neither gate can be bypassed by a default value — a missing or pending consent status blocks packaging outright, and a low quality score routes the episode to human review rather than silently downgrading its recommended use. This mirrors, and is stricter than, EgoInfinity's sanity-filter philosophy of flagging rather than silently altering questionable data.

8.3 Recommended-use tiers

- **Fine-tuning / direct policy training** — highest certificate tier: real ground-truth channels present, high perception confidence, retargeting-eligible.
- **Pretraining only** — usable for representation learning but not held to the same contact/kinematic precision bar.
- **Quarantined / needs review** — certificate below floor, or an unresolved sensor disagreement (e.g., vision says grasped, encoder says open) — never silently resolved one way.

9 Layer 5 — Cross-Embodiment Retargeting Engine

This is the layer that converts an agent-agnostic 4D representation into motion a specific customer's robot can actually execute. The design follows EgoInfinity's functional-retargeting philosophy directly: rather than requiring exact human body-pose recovery (which is often unavailable — a UMI or head-mounted clip may show only hands and a partial arm), the engine estimates a feasible robot-specific root transformation and preserves task-relevant end-effector motion within the target robot's own kinematic constraints.

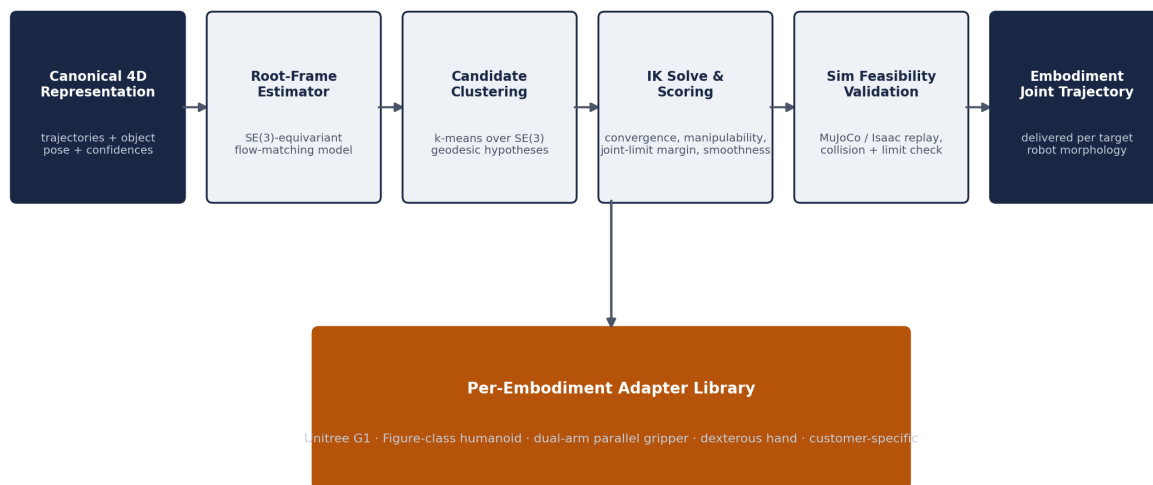


Figure 9.1 — The retargeting pipeline: canonical representation to root-frame estimation, candidate scoring, simulation validation, and per-embodiment delivery.

9.1 SE(3)-equivariant root-frame estimator

A simulation-trained neural estimator predicts a shared kinematic root frame (e.g., a torso frame for a humanoid, a base frame for a fixed-arm setup) conditioned on the recovered bilateral end-effector trajectories and the Layer 1 gravity estimate. The network is built from Vector-Neuron layers for exact rotation-equivariance, trained with a flow-matching objective rather than deterministic regression — this matters specifically because the same end-effector motion is consistent with multiple plausible torso/root poses under partial-body observation, and a single deterministic answer would silently discard that ambiguity rather than let downstream scoring resolve it.

9.2 Per-embodiment adapter library

Each target robot morphology (a specific humanoid, a dual-arm fixed-base setup, a dexterous hand, or a customer-proprietary design) is a separately trained root-frame estimator plus a kinematic model for IK — an *embodiment adapter*. New embodiments are onboarded by procedurally generating paired hand-trajectory / ground-truth-root-pose training data in simulation for that specific robot, exactly as EgoInfinity trains per-robot estimators, then registering the adapter in this library. This is the mechanism by which a new customer's specific robot becomes a supported delivery target without re-architecting the pipeline.

9.3 Candidate selection and IK scoring

At inference, multiple root-frame hypotheses are sampled and clustered; each candidate is scored by running the full-trajectory IK solve under that candidate as reference and evaluating convergence rate, residual tracking error, manipulability, joint-limit margin, and trajectory smoothness. The highest-scoring candidate is selected, blended with per-frame estimates to preserve genuine root motion (e.g., a person walking during a demonstration), and smoothed.

9.4 Simulation feasibility validation

Before any retargeted trajectory is marked eligible for delivery, it is replayed in a physics simulator against the target embodiment's actual kinematic and collision model. This is not optional polish — it is the concrete, checkable claim underlying the retargeting-eligibility field in the Layer 4 certificate. A trajectory that fails simulation replay (collision, joint limit violation, IK non-convergence) does not ship as retargeting-eligible for that embodiment, even if it ships successfully for a different one.

9.5 Finger-level retargeting for dexterous end-effectors

For embodiments with dexterous hands, arm joints follow the wrist-level IK target from 9.3, while finger joints are retargeted separately and directly from the recovered hand keypoints (or, on sensor-equipped rigs, from real glove joint angles) via a geometry-based, robot-specific finger mapping — arm-level and finger-level retargeting are deliberately decoupled, since they are governed by different kinematic constraints.

10 Layer 6 — Language & Task Annotation

Every episode is annotated at three levels of granularity, matching the depth expected by modern VLA training pipelines and by formal data-delivery specifications this pipeline is designed to satisfy.

10.1 Episode-level summary

A structured record per episode: task family, task name, a natural-language task description, scene description, success/failure with failure reason where applicable, and a canonical object list. Object references throughout all annotation layers are constrained to this canonical list — free-form object naming is not permitted, to keep annotations internally consistent and auditable against the video.

10.2 Subtask segmentation

Each episode is divided into coherent subtasks with start/end timestamps and frame indices, generated by a vision-language model grounded in actual sampled frames from the episode — not filled into a fixed template. Subtask descriptions are held to a concreteness bar: a description must be specific enough to audit directly against the video ("the right gripper lifts the red cube, transports it above the bin, and releases it" — not "handles object").

10.3 Fine-grained action intervals and fixed ontology

Beneath each subtask, fine-grained action intervals use a closed, fixed action vocabulary (reach, grasp, hold, lift, transport, lower, place, align, stabilize, release, open, close, insert, remove, push, pull, rotate, wipe, pour, idle) rather than free-form labels — this closed-vocabulary constraint is deliberately different from the open-vocabulary task-naming in 10.1: task *identity* should be open and descriptive, but atomic *action primitives* should be closed and standardized, since that is what makes cross-episode and cross-dataset comparison possible. Each action-interval row is joinable to the corresponding video frames, pose samples, and sensor windows it was derived from, and carries its own confidence score.

10.4 Actor labeling for bimanual and multi-actor scenes

Every action-interval row is attributed to a specific actor (left/right gripper, left/right hand, left/right fingers, controller, head, torso, body), constrained to actors for which a real tracking stream exists for that rig. Concurrent bimanual actions are represented as separate, potentially overlapping rows rather than collapsed into one — a session where the left and right end-effectors perform different primitives simultaneously must expose both.

10.5 Hallucination and consistency checking

Every language-model-generated field is checked against the structured facts and sampled frames it was conditioned on: references to objects absent from the canonical list, or actions not supported by the underlying primitive/sensor evidence, are flagged for review rather than silently accepted. This check is treated as seriously as the perceptual confidence checks in Layer 4 — a fluent but ungrounded description is exactly as untrustworthy as a low-confidence pose estimate.

11 Layer 7 — Dataset Packaging & Delivery

Packaging is deliberately the last layer to touch a customer-specific concern. Every layer before this one operates on the rig-agnostic canonical representation; only here does the pipeline branch by output format.

11.1 Structured delivery formats

- **LeRobot v3** — chunked structured data, synchronized video assets, and episode metadata compatible with current LeRobot dataset organization.
- **RLDS** — for customers whose training stack expects the Open X-Embodiment convention.
- **Raw provenance (MCAP)** — original per-episode raw capture, channel manifest, and a mapping from raw recording IDs to structured episode IDs, delivered alongside the structured package as required provenance, not as an optional extra.
- **Customer-specific schema adapters** — a thin export adapter per major customer format, reading only from the canonical representation of Layer 3 plus the certificate of Layer 4, so a new customer format never requires touching perception or retargeting code.

11.2 Manifests

Three manifest levels are generated for every delivery: an episode manifest (per-episode identifiers, duration, success/failure, stream inventory, canonical object list, known issues), a stream manifest (per-channel sample rate, timestamp basis, units, coordinate frame, calibration reference), and a dataset-level manifest (session/episode counts, task distribution, mean certificate scores, modality inventory).

11.3 Dataset-level quality operations

- **Deduplication** — cross-episode and cross-session near-duplicate detection via embedding similarity, applied before splits are generated so near-duplicates cannot leak across train/val/test.
- **Diversity and balance reporting** — object-class diversity, task-distribution imbalance ratio, environment/operator diversity where applicable.
- **Stratified splits** — train/val/test generated at the episode level, stratified by task, with dedup results consulted first.

11.4 Consent and provenance as delivery-blocking, not advisory

Consistent with Layer 4's fail-closed gate: the packaging stage itself re-verifies consent status at the moment of delivery, not only at the moment of initial processing — a session whose consent was revoked between processing and delivery must not ship, even if it was processed and certified earlier.

12 Layer 8 — Feedback Loop & Continuous Improvement

A modular, component-replaceable engine (Section 2) is only actually improvable if there is a concrete mechanism for knowing which components need replacing, and for propagating an upgrade across historical data without re-capturing anything.

12.1 Human-in-the-loop review routing

Episodes quarantined by Layer 4, or flagged by the Layer 6 hallucination check, or exhibiting a sensor-vs-vision disagreement in Layer 2, are routed to a review queue rather than silently resolved. Reviewer decisions (accept, correct, discard) are captured as structured feedback, not just a binary approval — corrected labels feed back into threshold and, eventually, model tuning.

12.2 Component versioning and reprocessing lineage

Every perception, refinement, and retargeting component carries an explicit version identifier, recorded per output artifact — not one blanket pipeline version. When a component is upgraded (a stronger hand-mesh model, an improved depth estimator), the set of historical sessions whose outputs depended on the old version is queryable directly from this lineage, making selective reprocessing of only the affected data tractable rather than requiring a full corpus re-run.

12.3 Calibration drift monitoring

Per-device calibration (Section 4.4) is periodically re-validated against fresh reference captures; a device whose effective calibration has drifted beyond tolerance is flagged, and sessions captured after the drift point are marked with reduced calibration-completeness confidence in their Layer 4 certificate until recalibration is confirmed.

13 Infrastructure & Scaling

13.1 Compute topology

Perception (Layer 1) and retargeting inference (Layer 5) are GPU-bound and stateless per episode — these run as horizontally scalable worker pools behind a job queue, with episode processing parallelized freely once ingestion (Layer 0) has produced the canonical raw container. Ingestion itself, and the certificate/packaging layers, are largely CPU-bound and run on cheaper compute, keeping GPU spend proportional only to the stages that actually need it.

13.2 Job orchestration

A queue-based worker architecture, matching the pattern validated in this pipeline's own service layer: jobs are submitted, tracked through a persistent status record (not only in-memory), and safely resumable — a worker restart mid-run must not lose track of which stages already completed for a given session.

13.3 Storage

- Raw provenance (MCAP containers): cold, high-durability object storage — written once, read rarely, kept indefinitely for reprocessing lineage.
- Canonical representation and intermediate perception outputs: warm storage, read frequently during retargeting and annotation.
- Delivered packages: versioned, customer-scoped storage with signed, time-limited access.

13.4 Cost proportionality

The two-pass processing pattern (Section 5.4) and the layered architecture together ensure compute cost scales with actual manipulation content and requested delivery formats, not with raw capture volume or the number of customer-specific export adapters in the system.

14 Comparative Positioning

Dimension	EgoInfinity	Open X-Embodiment / DROID	This pipeline
Data source	Internet video	Teleoperated robots	Multi-rig controlled capture (5 types)
Ground-truth sensing	None	Full (native robot)	Partial-to-full, per rig type, sensor-fused
Scale driver	Corpus size	Collection cost/hardware	Capture program throughput
Retargeting	Functional, cross-embodiment	N/A (native to one robot)	Functional, cross-embodiment, sim-validated
Certification	Sanity checks only	Not standardized	Formal per-episode certificate, fail-closed
Delivery format	Research browser + raw output	RLDS-family	Pluggable — LeRobot v3 / RLDS / customer schema
Consent/PII handling	Not addressed	Not addressed	Fail-closed gate, delivery-blocking

The practical implication: this pipeline is positioned to deliver something neither reference architecture targets directly — commercially certified, multi-rig, sensor-fused, retargeting-validated data, ready to train on without customer-side processing. EgoInfinity proves the underlying techniques are sound; Open-X/DROID prove the market values this data category; this architecture is the synthesis aimed at production delivery.

15 Appendix — Schemas, Thresholds & Coordinate Conventions

A.1 Consolidated tunable parameters

Parameter	Purpose
sync_drift_threshold_ms	Maximum allowed cross-stream timestamp drift before sync-integrity flag fires
grasp_trust_priority	Ordered list: hardware-confirmed > vision-2D-overlap > vision-3D-fallback
scale_sanity_factor	Maximum allowed ratio between monocular and mask-implied object scale before override
spurious_distance_m / spurious_2d_motion_px	Thresholds for flagging a tracked object as a likely background false match
retarget_ik_convergence_floor	Minimum per-frame IK success rate for an embodiment to be marked retargeting-eligible
cert_quarantine_floor	Minimum composite certificate score for an episode to be delivery-eligible at all
dedup_similarity_threshold	Embedding similarity above which two episodes are flagged as near-duplicates

A.2 Coordinate convention

Right-handed camera-world frame (+x right, +y down, +z into scene), gravity-aligned per session. Object-canonical points map to camera frame via a fixed scale-and-rotation convention compatible with standard segmentation/reconstruction tooling output conventions.

A.3 Ground-truth provenance enum

Every field in the Layer 3 canonical representation carries one of: `measured_hardware`, `estimated_vision_primary`, `estimated_vision_fallback`, `approximated` — this single field is what every downstream confidence computation, from the Layer 4 certificate through customer-facing delivery documentation, is ultimately keyed on.

End of document. This is a living architecture reference — component versions, thresholds, and embodiment adapters are expected to evolve as described in Section 12, without requiring changes to this document's layer boundaries.